HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CƠ BẢN I

THỰC TẬP CƠ SỞ

BÁO CÁO TUẦN

Đề tài:

Xây dựng website tuyển dụng việc làm

Giảng viên hướng dẫn	**: Kim Ngọc Bách**
Họ và tên sinh viên	**: Đỗ Bình Minh**
Ngày sinh	**: 24/12/2005**
Số điện thoại	**: 0329279863**
Mã sinh viên	**: B23DCCN542**
Lớp	**: D23CQCN10-B**
Nhóm lớp	**: 11**

Hà Nội – 2026

Mục lục

1. Tóm tắt các công việc đã thực hiện trong tuần 3
2. Chi tiết các công việc thực hiện trong tuần. 3
2.1. Tính năng phân trang 3
2.1.1. Xử lý bên phía Frontend 3
2.1.2. Xử lý bên phía backend 5
2.1.3. Kết quả 8
2.2. Tính năng sửa công việc 9
2.2.1. Xử lý bên phía Frontend 9
2.2.2. Xử lý bên phía Backend 16
2.2.3. Kết quả 20
2.3. Tính năng xóa công việc 21
2.3.1. Xử lý bên phía Frontend 21
2.3.2. Xử lý bên phía Backend 22
2.3.3. Kết quả 24
2.4. Khối nhà tuyển dụng hàng đầu 25
2.4.1. Xử lý bên phía Frontend 25
2.4.2. Xử lý bên phía Backend 26
2.4.3. Kết quả 29
2.5. Khối tìm kiếm 30
2.5.1. Xử lý phía Frontend 30
2.5.2. Kết quả 31
3. Kết quả đạt được 31
4. Khó khăn gặp phải 32
5. Hướng xử lý 32
6. Kế hoạch tuần tiếp theo 33

1. Tóm tắt các công việc đã thực hiện trong tuần

Trong tuần này, em tiếp tục hoàn thiện các chức năng quản lý công việc cho nhà tuyển dụng và bổ sung một số thành phần giao diện ở trang chủ. Cụ thể, em đã xây dựng tính năng phân trang danh sách công việc, chức năng sửa công việc và chức năng xóa công việc để nhà tuyển dụng có thể quản lý bài đăng thuận tiện hơn.

Bên cạnh đó, em cũng phát triển khối tìm kiếm việc làm trên trang chủ, cho phép người dùng nhập từ khóa và chọn thành phố để chuyển sang trang kết quả tìm kiếm. Ngoài ra, em xây dựng thêm khối "Nhà tuyển dụng hàng đầu", trong đó dữ liệu công ty được lấy từ backend thông qua API và hiển thị ra giao diện dưới dạng các thẻ công ty.

2. Chi tiết các công việc thực hiện trong tuần.

2.1. Tính năng phân trang

2.1.1. Xử lý bên phía Frontend

```
        <select
          name=""
          className="border border-[#DEDEDE] rounded-[8px] py-[12px] px-[18px] font-[400] text-[16px] text-
[#414042]"
          onChange={handlePagination}
        >
          {Array(totalPage).fill("").map((item, index) => (
            <option key={index} value={index+1}>
              Trang {index+1}
            </option>
          ))}
        </select>
```

Đoạn code này tạo **ô chọn số trang** trong React để phân trang.

Giải thích:

- <select>: tạo hộp chọn dạng dropdown.
- className="...": dùng để CSS giao diện, ví dụ viền, bo góc, padding, font, màu chữ.
- onChange={handlePagination}: khi người dùng chọn trang khác, hàm handlePagination sẽ được gọi.
- Array(totalPage).fill(""): tạo một mảng có số phần tử bằng tổng số trang.
- .map((item, index) => (...)): duyệt qua từng phần tử để tạo ra từng <option>.
- value={index + 1}: giá trị của option là số trang, bắt đầu từ 1.
- Trang {index + 1}: nội dung hiển thị cho người dùng.

```
const handlePagination = (event: any) => {
  const value = event.target.value;
  setPage(parseInt(value));
}
```

Đoạn code này là hàm xử lý khi người dùng chọn trang trong <select>

Giải thích:

- handlePagination: hàm xử lý sự kiện đổi trang.
- event: sự kiện được tạo ra khi người dùng chọn một option trong <select>.
- event.target.value: lấy giá trị của option đang được chọn.
- value lúc này thường là chuỗi, ví dụ "1", "2", "3".
- parseInt(value): chuyển chuỗi thành số nguyên.
- setPage(...): cập nhật state page.

```
useEffect(() => {
    fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/job/list?page=${page}`, {
      method: "GET",
      credentials: "include", // Gửi kèm cookie
    })
      .then(res => res.json())
      .then(data => {
        if(data.code == "success") {
          setJobList(data.jobs);
          setTotalPage(data.totalPage);
        }
      })
  }, [page]);
```

Đoạn code này dùng useEffect để **gọi API lấy danh sách job theo từng trang**.

Giải thích:

- useEffect(() => { ... }, [page]): mỗi khi biến page thay đổi thì chạy lại đoạn code bên trong.
- fetch(...): gọi API lấy danh sách công việc.
- page=${page}: gửi số trang hiện tại lên API.
- method: "GET": dùng phương thức GET để lấy dữ liệu.
- credentials: "include": gửi kèm cookie, thường dùng khi API cần xác thực đăng nhập.

- .then(res => res.json()): chuyển phản hồi từ API sang JSON.
- if (data.code == "success"): kiểm tra API trả về thành công.
- setJobList(data.jobs): lưu danh sách job vào state.
- setTotalPage(data.totalPage): lưu tổng số trang để hiển thị dropdown phân trang.

Luồng hoạt động:

1. Trang ban đầu có page = 1.
2. useEffect gọi API lấy job trang 1.
3. Người dùng chọn Trang 2.
4. setPage(2) làm page thay đổi.
5. useEffect chạy lại và gọi API với ?page=2.

2.1.2. Xử lý bên phía backend

```
export const listJob = async (req: AccountRequest, res: Response) => {
  try {
    const companyId = req.account.id;
    const find = {
      companyId: companyId
    }

    // Phân trang
    const limitItems = 2;
    let page = 1;
    if(req.query.page && parseInt(`${req.query.page}`) > 0) {
      page = parseInt(`${req.query.page}`);
    }
    const skip = (page - 1) * limitItems;
    const totalRecord = await Job.countDocuments(find);
    const totalPage = Math.ceil(totalRecord/limitItems);
    // Hết Phân trang

    const jobs = await Job
      .find(find)
      .sort({
        createdAt: "desc"
      })
      .limit(limitItems)
      .skip(skip)

    const dataFinal = [];
    res.json({
      code: "success",
      message: "Thành công!",
      jobs: dataFinal,
      totalPage: totalPage
    })
  } catch (error) {
    console.log(error);
```

Đây là API backend dùng để **lấy danh sách job của công ty đang đăng nhập**, có phân trang.

```
export const listJob = async (req: AccountRequest, res: Response) => {
```

Hàm listJob xử lý request từ frontend gửi lên.

```
const companyId = req.account.id;
```

Lấy id của công ty đang đăng nhập từ token/account.

```
const find = {
  companyId: companyId
}
```

Tạo điều kiện tìm kiếm: chỉ lấy các job thuộc công ty này.

```
const limitItems = 2;
let page = 1;
```

Mỗi trang hiển thị 2 job. Mặc định đang ở trang 1.

```
if (req.query.page && parseInt(`${req.query.page}`) > 0) {
  page = parseInt(`${req.query.page}`);
}
```

Nếu frontend gửi lên ?page=2, thì backend lấy page đó. Nếu không có thì vẫn là trang 1.

```
const skip = (page - 1) * limitItems;
```

Tính số bản ghi cần bỏ qua.

Ví dụ:

```
page = 1 => skip = 0
page = 2 => skip = 2
page = 3 => skip = 4
```

```
const totalRecord = await Job.countDocuments(find);
const totalPage = Math.ceil(totalRecord / limitItems);
```

Đếm tổng số job của công ty, rồi tính tổng số trang.

Ví dụ có 5 job, mỗi trang 2 job:

```
totalPage = Math.ceil(5 / 2) = 3
```

```
const jobs = await Job
  .find(find)
```

```
  .sort({ createdAt: "desc" })
  .limit(limitItems)
  .skip(skip);
```

Lấy danh sách job:

- .find(find): chỉ lấy job của công ty hiện tại.
- .sort({ createdAt: "desc" }): job mới nhất lên trước.
- .limit(limitItems): mỗi lần lấy 2 job.
- .skip(skip): bỏ qua các job của trang trước.

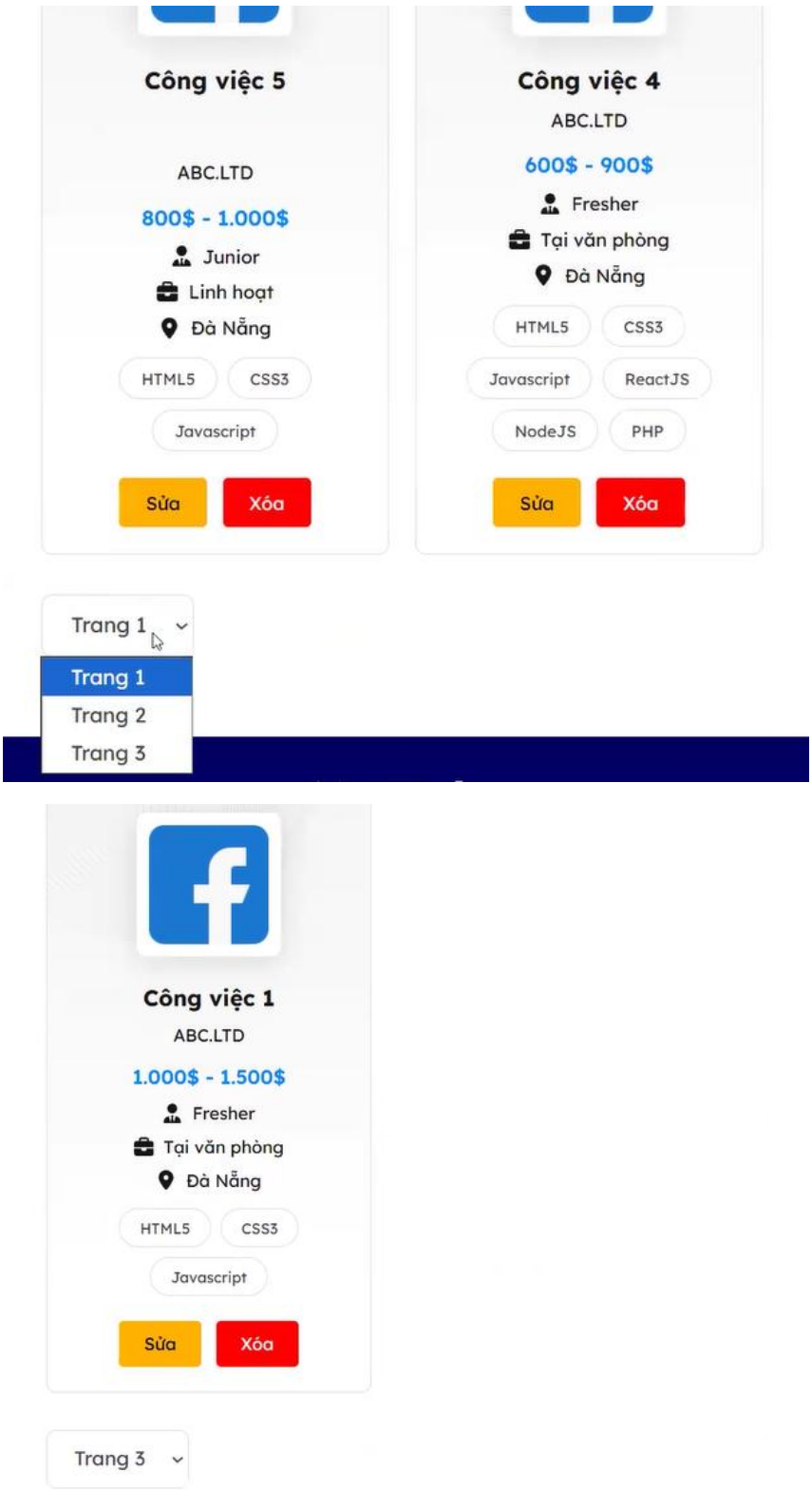
```
res.json({
  code: "success",
  message: "Thành công!",
  jobs: dataFinal,
  totalPage: totalPage
})
```

Trả dữ liệu về frontend gồm:

- code: trạng thái thành công.
- message: thông báo.
- jobs: danh sách job sau khi xử lý.
- totalPage: tổng số trang để frontend tạo dropdown phân trang.

Tóm lại: frontend gửi page, backend dựa vào page để lấy đúng nhóm job tương ứng rồi trả về danh sách job và tổng số trang.

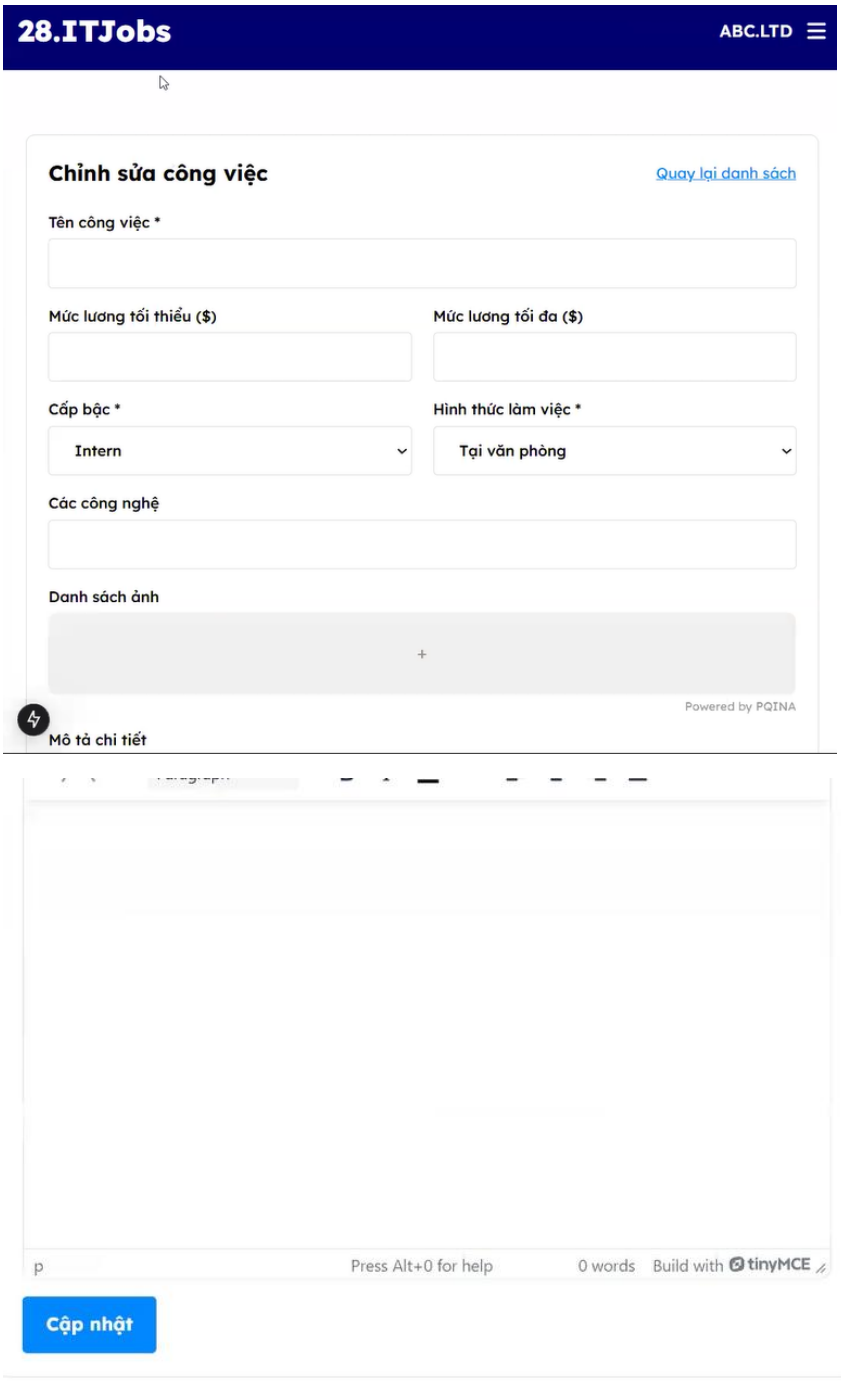
2.1.3. Kết quả



Tính năng phân trang hoàn thành với số trang hiển thị linh hoạt theo số công việc.

2.2. Tính năng sửa công việc

2.2.1. Xử lý bên phía Frontend



- Đây là giao diện trang chỉnh sửa công việc

```
registerPlugin(
  FilePondPluginFileValidateType,
  FilePondPluginImagePreview
);
```

Đoạn code này dùng để **đăng ký plugin cho FilePond**.

Giải thích:

- registerPlugin(...): hàm dùng để kích hoạt các plugin bổ sung cho FilePond.
- FilePondPluginFileValidateType: plugin kiểm tra loại file được upload, ví dụ chỉ cho phép ảnh .jpg, .png, .jpeg.
- FilePondPluginImagePreview: plugin hiển thị ảnh xem trước sau khi người dùng chọn file.

Tóm lại: đoạn này giúp FilePond **kiểm tra file upload có đúng định dạng không** và **hiển thị preview ảnh trước khi tải lên**.

```
export const FormEdit = (props: {
  id: string
}) => {
  const { id } = props;

  const editorRef = useRef(null);
  const [images, setImages] = useState<any[]>([]);
  const [jobDetail, setJobDetail] = useState<any>();
```

Đoạn code này là phần đầu của component FormEdit, dùng để **sửa thông tin job** theo id đồng thời khai báo các Ref và State

```
useEffect(() => {
  fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/job/edit/${id}`, {
    method: "GET",
    credentials: "include", // Gửi kèm cookie
  })
    .then(res => res.json())
    .then(data => {
      if(data.code == "error") {
        toast.error(data.message);
      }

      if(data.code == "success") {
        setJobDetail(data.jobDetail);
      }
    })
}, [id]);
```

Đoạn code này dùng để **gọi API lấy chi tiết job cần chỉnh sửa**.

Giải thích:

- useEffect(...): chạy đoạn code khi component được render hoặc khi id thay đổi.
- /company/job/edit/${id}: API lấy thông tin job theo id.
- method: "GET": gửi request dạng GET để lấy dữ liệu.
- credentials: "include": gửi kèm cookie đăng nhập để backend biết công ty nào đang đăng nhập.
- .then(res => res.json()): chuyển dữ liệu API trả về thành JSON.
- if (data.code == "error"): nếu API báo lỗi thì hiện thông báo lỗi bằng toast.error.
- if (data.code == "success"): nếu thành công thì lưu thông tin job vào state jobDetail.
- [id]: nếu id thay đổi thì gọi lại API để lấy job khác.

Tóm lại: đoạn này lấy dữ liệu job từ backend rồi đổ vào jobDetail để hiển thị lên form chỉnh sửa.

```
useEffect(() => {
  if(jobDetail) {
    // Hiển thị images mặc định
    if(jobDetail.images && jobDetail.images.length > 0) {
      const listImage = jobDetail.images.map((image: string) => {
        return {
          source: image
        };
      })
      setImages(listImage);
    }

    const validator = new JustValidate("#editForm");

    validator
      .addField('#title', [
        {
          rule: 'required',
          errorMessage: 'Vui lòng nhập tên công việc!'
        },
      ])
      .addField('#salaryMin', [
        {
          rule: 'minNumber',
          value: 0,
          errorMessage: 'Vui lòng nhập mức lương >= 0'
        },
      ])
      .addField('#salaryMax', [
        {
          rule: 'minNumber',
          value: 0,
          errorMessage: 'Vui lòng nhập mức lương >= 0'
        },
      ])
  }
}, [jobDetail]);
```

Đoạn code này chạy khi jobDetail có dữ liệu, dùng để **hiển thị ảnh cũ và tạo validate cho form sửa job**.

```
useEffect(() => {
  if (jobDetail) {
```

Chỉ chạy phần bên trong khi đã có dữ liệu jobDetail.

```
if (jobDetail.images && jobDetail.images.length > 0) {
  const listImage = jobDetail.images.map((image: string) => {
    return {
      source: image
```

```
    };
  })

  setImages(listImage);
}
```

Phần này xử lý ảnh cũ của job:

- Kiểm tra job có ảnh hay không.
- Duyệt qua jobDetail.images.
- Chuyển mỗi ảnh thành dạng:

```
{
  source: image
}
```

Đây là format thường dùng để FilePond hiển thị ảnh mặc định.

Sau đó:

setImages(listImage);

Cập nhật state images để hiện các ảnh cũ trong form chỉnh sửa.

const validator = new JustValidate("#editForm");

Tạo validate cho form có id là editForm.

```
validator
  .addField('#title', [
    {
      rule: 'required',
      errorMessage: 'Vui lòng nhập tên công việc!'
    },
  ])
```

Kiểm tra ô title không được để trống.

```
.addField('#salaryMin', [
  {
    rule: 'minNumber',
    value: 0,
    errorMessage: 'Vui lòng nhập mức lương >= 0'
```

```
  },
])
```

Kiểm tra salaryMin phải lớn hơn hoặc bằng 0.

```
.addField('#salaryMax', [
  {
    rule: 'minNumber',
    value: 0,
    errorMessage: 'Vui lòng nhập mức lương >= 0'
  },
])
```

Kiểm tra salaryMax cũng phải lớn hơn hoặc bằng 0.

```
}, [jobDetail]);
```

Mỗi khi jobDetail thay đổi thì đoạn useEffect này chạy lại.

Tóm lại: đoạn này dùng để **đổ ảnh cũ vào FilePond** và **thiết lập kiểm tra dữ liệu cho form chỉnh sửa job**.

```
const handleSubmit = (event: any) => {
  event.preventDefault();
  const title = event.target.title.value;
  const salaryMin = event.target.salaryMin.value;
  const salaryMax = event.target.salaryMax.value;
  const position = event.target.position.value;
  const workingForm = event.target.workingForm.value;
  const technologies = event.target.technologies.value;
  let description = "";
  if (editorRef.current) {
    description = (editorRef.current as any).getContent();
  }

  // Tạo FormData
  const formData = new FormData();
  formData.append("title", title);
  formData.append("salaryMin", salaryMin);
  formData.append("salaryMax", salaryMax);
  formData.append("position", position);
  formData.append("workingForm", workingForm);
  formData.append("technologies", technologies);
  formData.append("description", description);

  // Images
  if(images.length > 0) {
    for (const image of images) {
      formData.append("images", image.file);
    }
  }
  // End Images

  fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/job/edit/${id}`, {
    method: "PATCH",
    body: formData,
    credentials: "include", // Gửi kèm cookie
  })
    .then(res => res.json())
    .then(data => {
      if(data.code == "error") {
        toast.error(data.message);
      }

      if(data.code == "success") {
        toast.success(data.message);
      }
    })
}
```

Đoạn code này là hàm **submit form chỉnh sửa job** và gửi dữ liệu lên backend.

```
const handleSubmit = (event: any) => {
  event.preventDefault();
```

Chặn hành vi reload trang mặc định khi submit form.

```
const title = event.target.title.value;
const salaryMin = event.target.salaryMin.value;
const salaryMax = event.target.salaryMax.value;
const position = event.target.position.value;
const workingForm = event.target.workingForm.value;
const technologies = event.target.technologies.value;
```

Lấy dữ liệu người dùng nhập trong form.

```
let description = "";
if (editorRef.current) {
  description = (editorRef.current as any).getContent();
}
```

Lấy nội dung mô tả công việc từ editor, ví dụ TinyMCE.

```
const formData = new FormData();
```

Tạo FormData để gửi dữ liệu dạng form, đặc biệt dùng được khi có upload ảnh/file.

```
formData.append("title", title);
formData.append("salaryMin", salaryMin);
formData.append("salaryMax", salaryMax);
formData.append("position", position);
formData.append("workingForm", workingForm);
formData.append("technologies", technologies);
formData.append("description", description);
```

Đưa các dữ liệu của job vào formData.

```
if (images.length > 0) {
  for (const image of images) {
    formData.append("images", image.file);
  }
}
```

Nếu có ảnh thì thêm từng ảnh vào formData với key là "images".

Lưu ý nhỏ: đoạn này phù hợp với **ảnh mới upload**. Nếu ảnh cũ chỉ có dạng { source: image } thì có thể không có image.file.

```
fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/job/edit/${id}`, {
  method: "PATCH",
  body: formData,
  credentials: "include",
})
```

Gửi request lên API backend để cập nhật job theo id.

- PATCH: cập nhật dữ liệu.
- body: formData: gửi dữ liệu form và ảnh.
- credentials: "include": gửi kèm cookie đăng nhập.

```
.then(res => res.json())
.then(data => {
  if (data.code == "error") {
    toast.error(data.message);
  }

  if (data.code == "success") {
    toast.success(data.message);
  }
})
```

Nhận kết quả từ backend:

- Nếu lỗi thì hiện thông báo lỗi.
- Nếu thành công thì hiện thông báo thành công.

2.2.2. Xử lý bên phía Backend

```
router.get(
  "/job/edit/:id",
  authMiddleware.verifyTokenCompany,
  companyController.editJob
)

router.patch(
  "/job/edit/:id",
  authMiddleware.verifyTokenCompany,
  upload.array("images", 8),
  companyController.editJobPatch
)
```

Đoạn code này khai báo **2 route backend cho chức năng chỉnh sửa job**.

```
router.get(
  "/job/edit/:id",
  authMiddleware.verifyTokenCompany,
  companyController.editJob
)
```

Route này dùng để **lấy chi tiết job cần chỉnh sửa**.

- GET /job/edit/:id: API lấy thông tin job theo id.
- :id: id của job trên URL.
- authMiddleware.verifyTokenCompany: kiểm tra công ty đã đăng nhập chưa.
- companyController.editJob: controller xử lý lấy dữ liệu job.

Ví dụ frontend gọi:

/company/job/edit/123

Backend sẽ lấy job có id là 123.

```
router.patch(
  "/job/edit/:id",
  authMiddleware.verifyTokenCompany,
  upload.array("images", 8),
  companyController.editJobPatch
)
```

Route này dùng để **cập nhật job sau khi sửa**.

- PATCH /job/edit/:id: API cập nhật job theo id.
- authMiddleware.verifyTokenCompany: kiểm tra quyền công ty.
- upload.array("images", 8): cho phép upload nhiều ảnh với field name là "images", tối đa 8 ảnh.
- companyController.editJobPatch: controller xử lý cập nhật dữ liệu.

```
export const editJob = async (req: AccountRequest, res: Response) => {
  try {
    const id = req.params.id;
    const companyId = req.account.id;

    const jobDetail = await Job.findOne({
      _id: id,
      companyId: companyId
    })

    if(!jobDetail) {
      res.json({
        code: "error",
        message: "Dữ liệu không hợp lệ!"
      })
      return;
    }

    res.json({
      code: "success",
      message: "Thành công!",
      jobDetail: jobDetail
    })
  } catch (error) {
    res.json({
      code: "error",
      message: "Dữ liệu không hợp lệ!"
    })
  }
}

export const editJobPatch = async (req: AccountRequest, res: Response) => {
  try {
    const id = req.params.id;
    const companyId = req.account.id;

    const jobDetail = await Job.findOne({
      _id: id,
      companyId: companyId
    })

    if(!jobDetail) {
      res.json({
        code: "error",
        message: "Dữ liệu không hợp lệ!"
      })
      return;
    }

    req.body.companyId = req.account.id;
    req.body.salaryMin = req.body.salaryMin ? parseInt(req.body.salaryMin) : 0;
    req.body.salaryMax = req.body.salaryMax ? parseInt(req.body.salaryMax) : 0;
    req.body.technologies = req.body.technologies ? req.body.technologies.split(", ") : [];
    req.body.images = [];

    // Xử lý mảng images
    if(req.files) {
      for (const file of req.files as any[]) {
        req.body.images.push(file.path);
      }
    }
    // Hết Xử lý mảng images

    await Job.updateOne({
```

```
    await Job.updateOne({
      _id: id,
      companyId: companyId
    }, req.body);

    res.json({
      code: "success",
      message: "Cập nhật thành công!"
    })
  } catch (error) {
    res.json({
      code: "error",
      message: "Dữ liệu không hợp lệ!"
    })
  }
```

Đoạn code này là **controller backend** cho chức năng chỉnh sửa job.

1. editJob

Dùng để **lấy chi tiết công việc cần sửa**.

const id = req.params.id;
const companyId = req.account.id;

Lấy id job từ URL và companyId từ tài khoản đang đăng nhập.

const jobDetail = await Job.findOne({
  _id: id,
  companyId: companyId
})

Tìm job theo id và phải thuộc đúng công ty đó.

Nếu không có:

message: "Dữ liệu không hợp lệ!"

Nếu có thì trả về:

jobDetail: jobDetail

2. editJobPatch

Dùng để **cập nhật công việc sau khi sửa**.

Nó cũng kiểm tra job có đúng công ty đang đăng nhập không:

_id: id,
companyId: companyId

Sau đó xử lý dữ liệu:

req.body.salaryMin = parseInt(req.body.salaryMin)
req.body.salaryMax = parseInt(req.body.salaryMax)

Chuyển lương từ chuỗi sang số.

req.body.technologies = req.body.technologies.split(", ")

Chuyển chuỗi công nghệ thành mảng.

req.body.images.push(file.path)

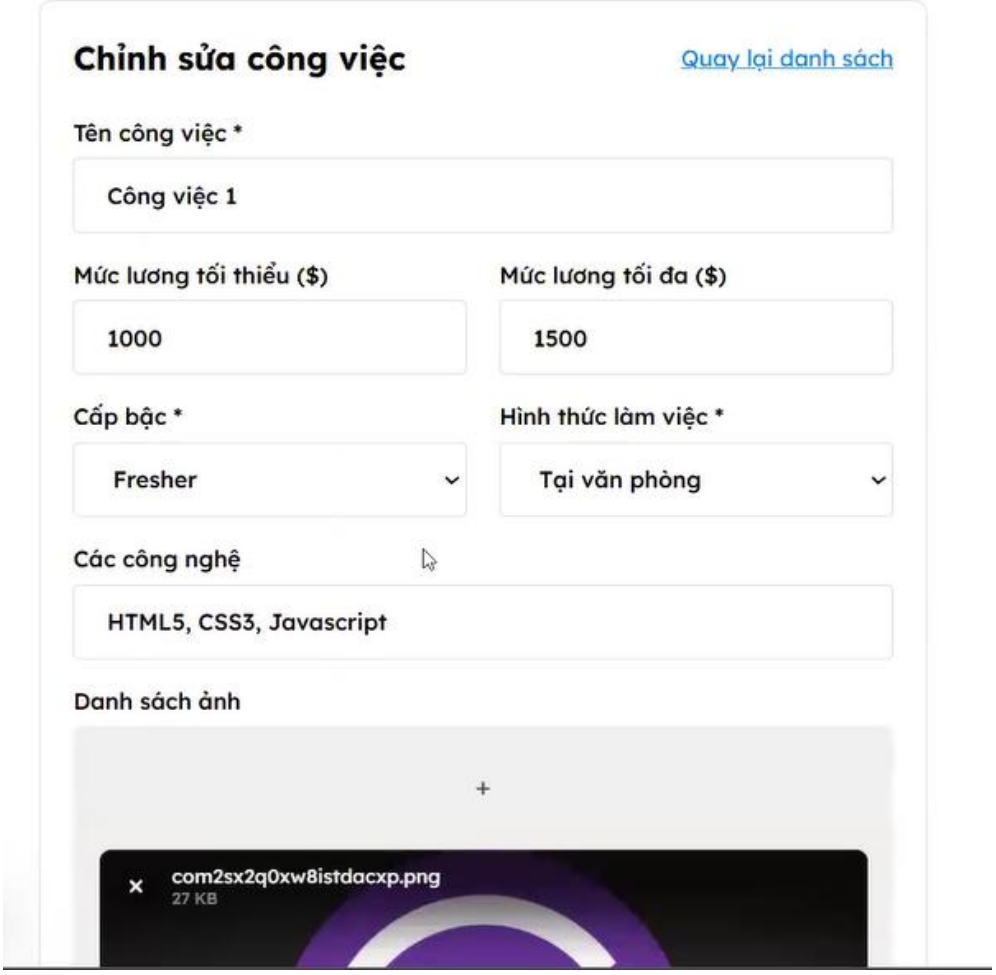
Lưu đường dẫn ảnh upload.

Cuối cùng:

await Job.updateOne({ _id: id, companyId }, req.body)

Cập nhật job trong database.

2.2.3. Kết quả



Trang đã hiển thị thông tin chi tiết của công việc và sẵn sàng trong việc chỉnh sửa

2.3. Tính năng xóa công việc

2.3.1. Xử lý bên phía Frontend

```
export const JobList = () => {
  const [jobList, setJobList] = useState<any[]>([]);
  const [page, setPage] = useState(1);
  const [totalPage, setTotalPage] = useState(0);
+ const [countDelete, setCountDelete] = useState(0);

  useEffect(() => {
    fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/job/list?page=${page}`, {
@@ -23,15 +25,34 @@ export const JobList = () => {
          setTotalPage(data.totalPage);
        }
      })
  }, [page, countDelete]);

  const handlePagination = (event: any) => {
    const value = event.target.value;
    setPage(parseInt(value));
  }

  const handleDelete = (id: string) => {
    fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/job/delete/${id}`, {
      method: "DELETE",
      credentials: "include", // Gửi kèm cookie
    })
      .then(res => res.json())
      .then(data => {
        if(data.code == "error") {
          toast.error(data.message);
        }

        if(data.code == "success") {
          toast.success(data.message);
          setCountDelete(countDelete + 1);
        }
      })
  }
```

Đoạn code này thêm chức năng **xóa job và tự load lại danh sách**.

const [countDelete, setCountDelete] = useState(0);

Tạo state đếm số lần xóa job.

```
useEffect(() => {
  fetch(`/company/job/list?page=${page}`)
    ...
}, [page, countDelete]);
```

useEffect sẽ gọi lại API lấy danh sách job khi:

- page thay đổi

- countDelete thay đổi

```
const handleDelete = (id: string) => {
  fetch(`/company/job/delete/${id}`, {
    method: "DELETE",
    credentials: "include",
  })
```

Hàm này gọi API xóa job theo id.

```
if (data.code == "success") {
  toast.success(data.message);
  setCountDelete(countDelete + 1);
}
```

Nếu xóa thành công:

- Hiện thông báo thành công.
- Tăng countDelete lên 1.

Khi countDelete thay đổi, useEffect chạy lại, nên danh sách job được cập nhật lại.

2.3.2. Xử lý bên phía Backend

```
router.delete(
  "/job/delete/:id",
  authMiddleware.verifyTokenCompany,
  companyController.deleteJobDel
)
```

Đoạn này khai báo route API để **xóa job**.

```
router.delete(
  "/job/delete/:id",
  authMiddleware.verifyTokenCompany,
  companyController.deleteJobDel
)
```

Giải thích ngắn gọn:

- DELETE /job/delete/:id: API xóa job theo id.
- :id: mã công việc cần xóa.
- authMiddleware.verifyTokenCompany: kiểm tra công ty đã đăng nhập chưa.
- companyController.deleteJobDel: controller xử lý xóa job.

```
export const deleteJobDel = async (req: AccountRequest, res: Response) => {
  try {
    const id = req.params.id;
    const companyId = req.account.id;

    const jobDetail = await Job.findOne({
      _id: id,
      companyId: companyId
    })

    if(!jobDetail) {
      res.json({
        code: "error",
        message: "Dữ liệu không hợp lệ!"
      })
      return;
    }

    await Job.deleteOne({
      _id: id,
      companyId: companyId
    })

    res.json({
      code: "success",
      message: "Đã xóa công việc!"
    })
  } catch (error) {
    res.json({
      code: "error",
      message: "Dữ liệu không hợp lệ!"
    })
  }
}
```

Đoạn này là controller backend để **xóa một công việc**.

const id = req.params.id;
const companyId = req.account.id;

Lấy:

- id: id công việc từ URL.
- companyId: id công ty đang đăng nhập.

const jobDetail = await Job.findOne({
  _id: id,
  companyId: companyId
})

Kiểm tra job có tồn tại và có thuộc công ty này không.

Nếu không có:

```
res.json({
  code: "error",
  message: "Dữ liệu không hợp lệ!"
})
```

Nếu có thì xóa:

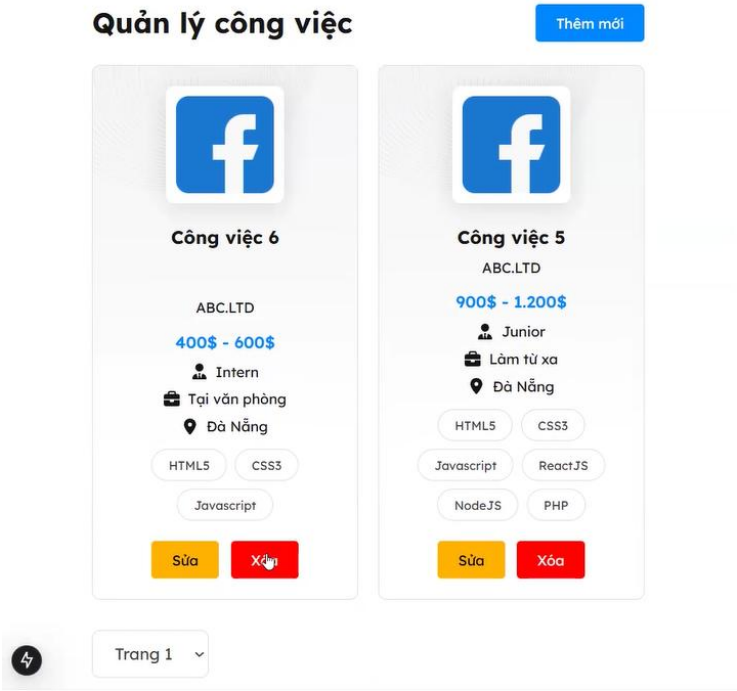
```
await Job.deleteOne({
  _id: id,
  companyId: companyId
})
```

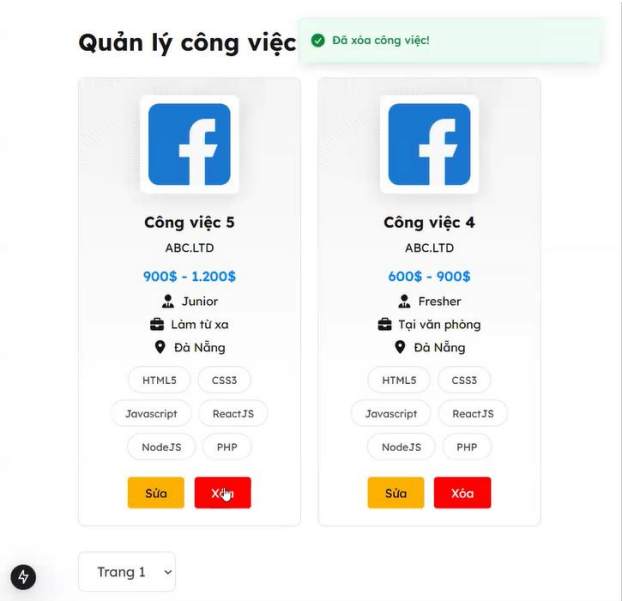
Cuối cùng trả về:

```
res.json({
  code: "success",
  message: "Đã xóa công việc!"
})
```

Tóm lại: controller này **kiểm tra đúng công ty sở hữu job**, sau đó mới cho phép xóa job đó.

2.3.3. Kết quả





Sau khi ấn nút xóa công việc, công việc đó sẽ biến mất và giao diện được render lại

2.4. Khối nhà tuyển dụng hàng đầu

2.4.1. Xử lý bên phía Frontend

```
"use client"
import { useEffect, useState } from "react";
import { CardCompanyItem } from "../card/CardCompanyItem"

export const Section2 = () => {
  const [companyList, setCompanyList] = useState<any[]>([]);

  useEffect(() => {
    fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/list?limitItems=9`)
      .then(res => res.json())
      .then(data => {
        if(data.code == "success") {
          setCompanyList(data.companyList);
        }
      })
  }, []);

  return (
    <>
      <div className="py-[60px]">
        <div className="container mx-auto px-[16px]">
          <h2 className="font-[700] sm:text-[28px] text-[24px] text-[#121212] text-center mb-[30px]">
            Nhà tuyển dụng hàng đầu
          </h2>
          {/* Wrap */}
          <div className="grid lg:grid-cols-3 grid-cols-2 sm:gap-[20px] gap-x-[10px] gap-y-[20px]">
            {/* Item */}
            {companyList.map((item) => (
              <CardCompanyItem key={item.id} {...item} />
            ))}
          </div>
        </div>
      </div>
    </>
  )
}
```

Đoạn code này tạo **Section hiển thị danh sách "Nhà tuyển dụng hàng đầu"** trên giao diện React/Next.js.

Cụ thể:

"use client"
→ Cho biết component này chạy ở phía client, vì có dùng useState, useEffect.

useState<any[]>([])
→ Tạo state companyList để lưu danh sách công ty lấy từ API.

useEffect(() => { ... }, [])
→ Chạy một lần khi component được render lần đầu.

fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/list?limitItems=9`)

→ Gọi API lấy danh sách công ty, giới hạn 9 công ty.

```
if(data.code == "success") {
  setCompanyList(data.companyList);
}
```

→ Nếu API trả về thành công thì lưu danh sách công ty vào companyList.

Phần return
→ Hiển thị giao diện gồm tiêu đề:

Nhà tuyển dụng hàng đầu

và danh sách công ty dạng lưới:

```
companyList.map((item) => (
  <CardCompanyItem key={item.id} {...item} />
))
```

→ Duyệt từng công ty trong companyList và truyền dữ liệu sang component CardCompanyItem để hiển thị từng thẻ công ty.

2.4.2. Xử lý bên phía Backend

```
router.get(
  "/list",
  companyController.list
)
```

Đoạn code này định nghĩa **route API** cho chức năng lấy danh sách công ty.

```
router.get(
  "/list",
  companyController.list
)
```

Ý nghĩa:

router.get("/list", ...)
→ Tạo API dạng **GET** với đường dẫn /list.

companyController.list
→ Khi client gọi API này, hàm list trong companyController sẽ được chạy để xử lý yêu cầu.

```
export const list = async (req: Request, res: Response) => {
  let limitItems = 12;
  if(req.query.limitItems) {
    limitItems = parseInt(`${req.query.limitItems}`);
  }

  const companyList = await AccountCompany
    .find({})
    .limit(limitItems)

  const companyListFinal = [];

  for (const item of companyList) {
    const dataItem = {
      id: item.id,
      logo: item.logo,
      companyName: item.companyName,
      cityName: "",
      totalJob: 0
    };

    // Thành phố
    const city = await City.findOne({
      _id: item.city
    })
    dataItem.cityName = `${city ? city.name : ""}`;

    // Tổng số việc làm
    const totalJob = await Job.countDocuments({
      companyId: item.id
    });
    dataItem.totalJob = totalJob;

    companyListFinal.push(dataItem);
  }

  res.json({
    code: "success",
    message: "Thành công",
    companyList: companyListFinal
  })
}
```

Đoạn code này là **hàm controller xử lý API lấy danh sách công ty**.

Tác dụng chính:

```
let limitItems = 12;
```

→ Mặc định lấy tối đa 12 công ty.

```
if(req.query.limitItems) {
  limitItems = parseInt(`${req.query.limitItems}`);
}
```

→ Nếu client truyền limitItems trên URL thì dùng số đó để giới hạn số công ty.

Ví dụ:

```
/company/list?limitItems=9
```

→ Chỉ lấy 9 công ty.

```
const companyList = await AccountCompany
  .find({})
  .limit(limitItems)
```

→ Lấy danh sách công ty từ database, giới hạn theo limitItems.

Trong vòng lặp:

```
for (const item of companyList) {
```

→ Duyệt từng công ty để tạo dữ liệu trả về.

```
const city = await City.findOne({
  _id: item.city
})
```

→ Tìm thành phố của công ty.

```
dataItem.cityName = `${city ? city.name : ""}`;
```

→ Gán tên thành phố vào dữ liệu công ty.

```
const totalJob = await Job.countDocuments({
  companyId: item.id
});
```

→ Đếm tổng số việc làm của công ty đó.

```
companyListFinal.push(dataItem);
```

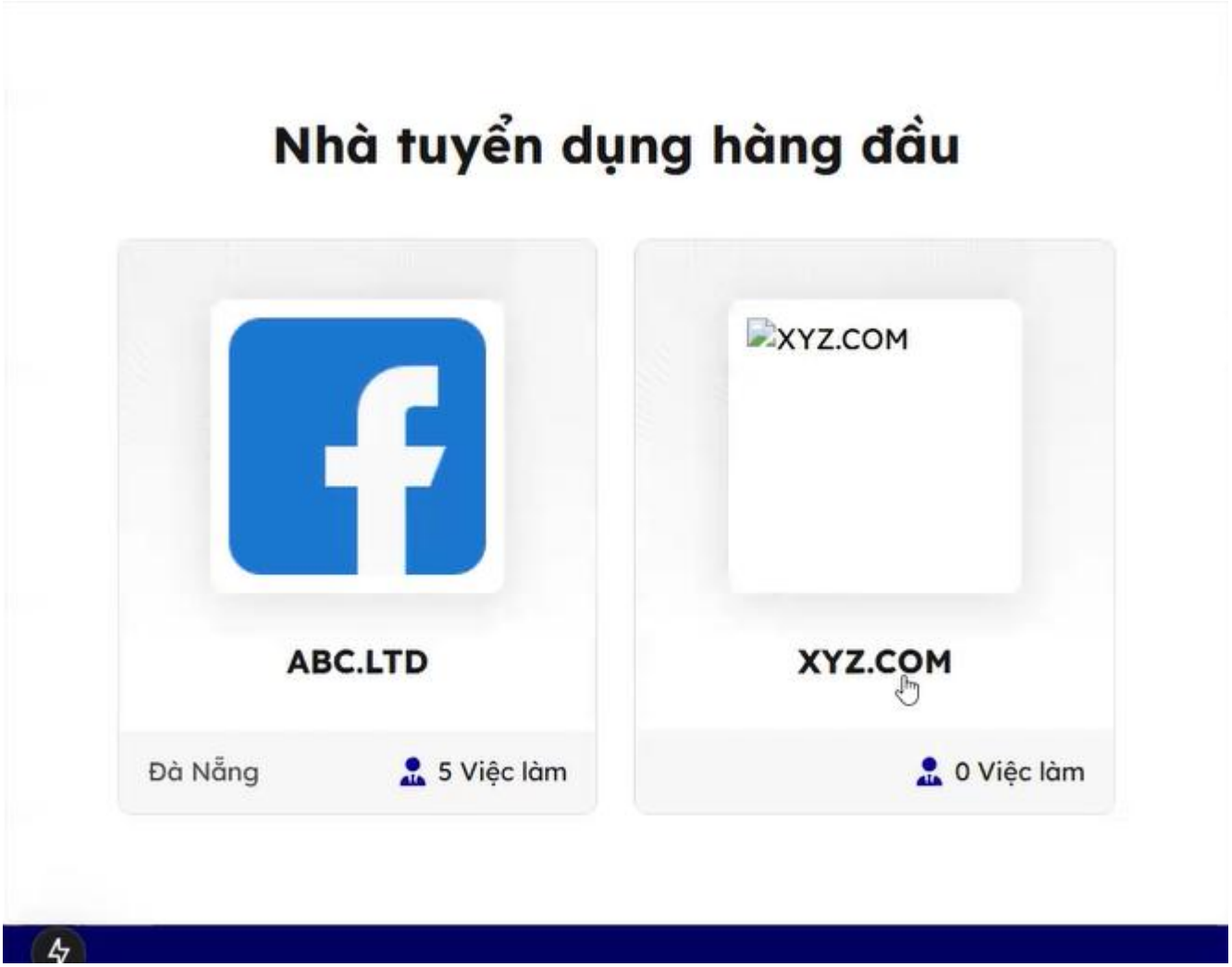
→ Thêm dữ liệu công ty đã xử lý vào danh sách kết quả.

Cuối cùng:

```
res.json({
  code: "success",
  message: "Thành công",
  companyList: companyListFinal
})
```

→ Trả kết quả JSON về cho frontend.

2.4.3. Kết quả



Các công ty tuyển dụng hàng đầu sẽ được hiển thị

2.5. Khối tìm kiếm

2.5.1. Xử lý phía Frontend

```
export const Section1 = () => {
  const router = useRouter();

  const handleSearch = (event: any) => {
    event.preventDefault();
    const city = event.target.city.value;
    const keyword = event.target.keyword.value;
    const link = `/search?city=${city}&keyword=${keyword}`;
    router.push(link);
  }
```

Đoạn code này xử lý **chức năng tìm kiếm việc làm** ở frontend.

const router = useRouter();

→ Tạo router để chuyển trang trong Next.js.

const handleSearch = (event: any) => {

→ Tạo hàm xử lý khi người dùng submit form tìm kiếm.

event.preventDefault();

→ Chặn việc form reload lại trang.

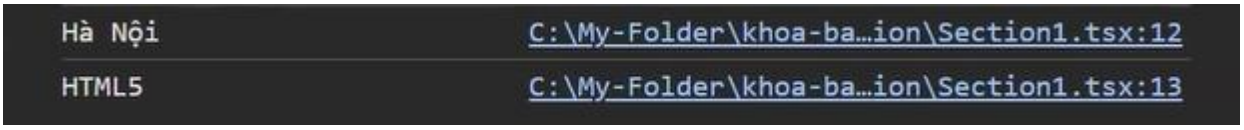
const city = event.target.city.value;
const keyword = event.target.keyword.value;

→ Lấy giá trị người dùng nhập/chọn từ ô city và keyword.

const link = `/search?city=${city}&keyword=${keyword}`;

→ Tạo đường dẫn tìm kiếm có chứa tham số trên URL.

2.5.2. Kết quả



Sau khi nhập thông tin và bấm tìm kiếm sẽ lấy ra được thông tin để tạo đường dẫn tìm kiếm có chứa tham số trên URL sau này.

3. Kết quả đạt được

Trong tuần này, em đã hoàn thiện thêm một số chức năng quan trọng cho website tuyển dụng việc làm.

Thứ nhất, em đã xây dựng được tính năng phân trang cho danh sách công việc, giúp dữ liệu được chia thành nhiều trang nhỏ thay vì hiển thị toàn bộ cùng lúc. Điều này giúp giao diện quản lý công việc gọn gàng và dễ theo dõi hơn.

Thứ hai, em đã hoàn thiện chức năng sửa công việc cho nhà tuyển dụng. Hệ thống có thể lấy thông tin công việc cần sửa, hiển thị lại dữ liệu lên form và gửi dữ liệu mới lên backend để cập nhật vào cơ sở dữ liệu.

Thứ ba, em đã xây dựng chức năng xóa công việc. Khi nhà tuyển dụng xóa một bài đăng, hệ thống gửi yêu cầu đến backend để xóa dữ liệu và cập nhật lại danh sách công việc trên giao diện.

Ngoài ra, em cũng hoàn thiện thêm khối tìm kiếm trên trang chủ. Người dùng có thể nhập từ khóa, chọn thành phố và được chuyển sang trang tìm kiếm thông qua đường dẫn có chứa tham số city và keyword.

Cuối cùng, em đã xây dựng khối "Nhà tuyển dụng hàng đầu". Dữ liệu công ty được lấy từ API backend, sau đó hiển thị ra giao diện dưới dạng các thẻ công ty, bao gồm tên công ty, logo, thành phố và tổng số công việc.

4. Khó khăn gặp phải

Trong quá trình thực hiện, em gặp khó khăn khi đồng bộ dữ liệu giữa frontend và backend, đặc biệt ở các chức năng phân trang, sửa và xóa công việc.

Với phân trang, cần xử lý đúng trang hiện tại, số lượng bản ghi mỗi trang và tổng số trang để dữ liệu hiển thị không bị sai hoặc trùng lặp.

Với chức năng sửa công việc, khó khăn là phải hiển thị lại chính xác dữ liệu cũ lên form, sau đó gửi dữ liệu mới đúng định dạng về backend.

Với chức năng xóa công việc, cần đảm bảo xóa đúng bài đăng được chọn và cập nhật lại giao diện sau khi xóa thành công.

Ngoài ra, ở khối tìm kiếm và khối nhà tuyển dụng hàng đầu, em cần xử lý đúng đường dẫn API, dữ liệu trả về và cách hiển thị dữ liệu lên giao diện.

5. Hướng xử lý

Để xử lý các khó khăn trên, em tách từng chức năng thành các phần nhỏ để kiểm tra riêng.

Với phân trang, em xử lý phía frontend để lưu trang hiện tại và gửi tham số lên backend. Backend sẽ giới hạn số lượng bản ghi trả về theo từng trang.

Với chức năng sửa công việc, em lấy dữ liệu chi tiết của bài đăng cần sửa, đưa dữ liệu đó vào form, sau đó gửi request cập nhật lên backend để lưu thay đổi vào cơ sở dữ liệu.

Với chức năng xóa công việc, em tạo API xóa riêng ở backend. Sau khi xóa thành công, frontend cập nhật lại danh sách công việc để giao diện hiển thị đúng dữ liệu mới.

Với khối tìm kiếm, em dùng useRouter để chuyển trang và truyền dữ liệu tìm kiếm qua URL. Với khối nhà tuyển dụng hàng đầu, em dùng useEffect để gọi API, lưu dữ liệu vào state và render danh sách công ty bằng component riêng.

6. Kế hoạch tuần tiếp theo

Trong tuần tiếp theo, em dự kiến tiếp tục hoàn thiện trang tìm kiếm và trang chi tiết công việc của website tuyển dụng việc làm. Trọng tâm là bổ sung thêm dữ liệu hiển thị và các bộ lọc để người dùng có thể tìm kiếm công việc thuận tiện hơn.

Cụ thể, em sẽ hoàn thiện việc hiển thị dữ liệu theo các trường như ngôn ngữ lập trình, thành phố, công ty và từ khóa tìm kiếm. Các phần này sẽ được xử lý ở cả frontend và backend để đảm bảo dữ liệu được lấy đúng từ API và hiển thị chính xác trên giao diện.

Bên cạnh đó, em sẽ xây dựng thêm các bộ lọc trong trang tìm kiếm, bao gồm bộ lọc theo cấp bậc và bộ lọc theo hình thức làm việc. Những bộ lọc này giúp người dùng thu hẹp kết quả tìm kiếm theo nhu cầu thực tế, ví dụ như tìm việc theo vị trí thực tập, nhân viên, trưởng nhóm hoặc theo hình thức làm việc toàn thời gian, bán thời gian, từ xa.

Ngoài ra, em sẽ tiếp tục phát triển tính năng phân trang cho trang kết quả tìm kiếm. Khi số lượng công việc nhiều, phân trang sẽ giúp dữ liệu được chia nhỏ, giao diện dễ theo dõi hơn và giảm tải khi hiển thị danh sách công việc.

Cuối cùng, em dự kiến xây dựng trang chi tiết công việc ở cả frontend và backend. Trang này sẽ cho phép người dùng xem đầy đủ thông tin của một công việc cụ thể như tên công việc, công ty tuyển dụng, địa điểm, mức lương, mô tả công việc, yêu cầu ứng viên và các thông tin liên quan khác.